

Parallélisme, Multithreading et Jobs d'Arrière-Plan en PowerShell

Ce document détaille les mécanismes d'exécution asynchrone et concurrente, le fonctionnement du multithreading moderne, le cycle de vie complet des tâches de fond (*Background Jobs*), ainsi que les règles d'optimisation indispensables pour diviser le temps de traitement des scripts d'infrastructure.

1. Paralléliser en Administration Système

1.1 La limite structurelle du modèle séquentiel

Par défaut, l'interpréteur PowerShell est **synchrone** et **mono-thread** : chaque commande s'exécute l'une après l'autre sur le thread principal de la console.

Dans une boucle séquentielle classique :

- Si une tâche unitaire prend 2sec (par exemple une vérification de port ou un ping avec timeout),
- Et que vous devez auditer un parc de 30 serveurs ou composants,

Pendant toute cette minute, la console PowerShell de l'administrateur est **totalelement gelée** : impossible de saisir une autre commande, d'afficher un retour visuel en temps réel ou d'interrompre proprement une tâche bloquée sans tuer le processus hôte.

1.2 Le paradigme asynchrone et concurrent

En exécution concurrente, PowerShell distribue les opérations sur plusieurs unités d'exécution simultanées (cœurs de processeur ou threads du système d'exploitation).

Pour nos 30 serveurs exécutés en parallèle avec un délai unitaire de 2secondes , le temps global de traitement passe d'une minute entière à **environ 3 à 4 secondes**.

2. Mesure Objective des Performances (Measure-Command)

Avant de chercher à optimiser un traitement, l'administrateur doit mesurer rigoureusement son temps d'exécution.

La cmdlet Measure-Command exécute un bloc de script {} en isolant précisément sa durée d'exécution sans polluer l'affichage console avec les objets produits.

```
$Chrono = Measure-Command {  
    1..5 | ForEach-Object {
```

```

        Start-Sleep -Seconds 1
    }
}

```

```

# Extraction du temps d'exécution en secondes
$Chrono.TotalSeconds

```

Propriétés clés d'un objet `TimeSpan` renvoyé par `Measure-Command` :

- **TotalSeconds** : Durée totale convertie en secondes (nombre décimal haute précision).
- **TotalMilliseconds** : Durée en millisecondes (indispensable pour les micro-benchmarks).
- **Minutes / Seconds** : Décomposition calendaire de la durée.

3. Le Multithreading Moderne : `ForEach-Object -Parallel` (PowerShell 7+)

Il permet d'exécuter des itérations simultanées **au sein du même processus hôte**, via des threads managés .NET légers.

3.1 Fonctionnement interne et mécanisme de Thread Pool

Contrairement au modèle classique séquentiel qui traite les éléments du pipeline un à un, `ForEach-Object -Parallel` injecte chaque élément dans le pool de threads (*ThreadPool*) du processus `powershell.exe` :

```

$Serveurs = @("SRV-APP01", "SRV-APP02", "SRV-APP03", "SRV-APP04", "SRV-APP05")

$Serveurs | ForEach-Object -ThrottleLimit 5 -Parallel {
    $ThreadId = [System.Threading.Thread]::CurrentThread.ManagedThreadId
    Write-Host "[$_] Audit en cours sur le Thread ID : $ThreadId" -ForegroundColor Cyan
    Start-Sleep -Seconds 2
}

```

Autre exemple : on demande à notre processeur de calculer des empreintes de fichiers en parallèle :

Le processeur local utilise plusieurs cœurs pour hacher 4 gros fichiers ISO en même temps

```

Get-Childitem "C:\ISOs\*.iso" | ForEach-Object -Parallel {
    Get-FileHash -Path $_.FullName -Algorithm SHA256
}

```

```
}
```

Ici, c'est le processeur de chaque serveur cible qui fait le calcul :

```
Invoke-Command -ComputerName "SRV-01", "SRV-02" -ScriptBlock {  
  
    Get-FileHash -Path "C:\Windows\explorer.exe"  
  
}
```

3.2 Injection de variables parentes : Le modificateur de portée \$using:

Puisque chaque thread tourne dans un Runspace isolé, les variables déclarées dans le script principal ne sont pas implicitement accessibles dans le bloc d'instructions parallèles.

Pour importer la valeur d'une variable locale dans le bloc d'exécution, on utilise impérativement le préfixe de portée **\$using** :

```
$SeuilAlerteMo = 500
```

```
Get-Content "C:\Audit\serveurs.txt" | ForEach-Object -Parallel {  
    # $SeuilAlerteMo n'existe pas ici sans $using:  
    $Limite = $using:SeuilAlerteMo  
    Get-Process | Where-Object WorkingSet64 -gt ($Limite * 1MB)  
}
```

3.3 Exécution non-bloquante avec le commutateur -AsJob

Par défaut, ForEach-Object -Parallel libère plusieurs threads mais **bloque la console** jusqu'à ce que le dernier élément de la collection soit traité.

Pour rendre l'opération entièrement asynchrone et récupérer immédiatement la main dans le shell, ajoutez le commutateur **-AsJob** :

```
# Démarre l'exécution multithreadée en tâche de fond  
$MonJobMultiThread = 1..20 | ForEach-Object -Parallel {  
    Start-Sleep -Seconds 2  
    $_ * 10  
} -AsJob
```

```
# La console est immédiatement disponible. On récupère les résultats quand on veut :  
$MonJobMultiThread | Receive-Job -Wait -AutoRemoveJob
```

3.4 Avantages et Inconvénients de ForEach-Object -Parallel

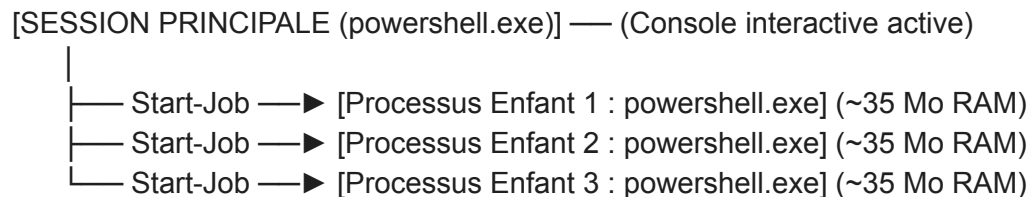
Caractéristique	Analyse technique
Avantages	1. Empreinte mémoire minime : Chaque thread ne consomme qu'environ 1 à 2 Mo de RAM. 2. Latence de démarrage quasi-nulle : Démarrage instantané (< 1 ms). 3. Syntaxe concise : Aucun cycle de vie complexe à gérer manuellement (Wait, Receive, Purge). 4. Pipeline fluide : Les objets retournés sortent directement dans le flux standard sans étape de désérialisation lourde. 5. Régulation native : Contrôle précis de la concurrence via le paramètre dédié -ThrottleLimit.
Inconvénients	1. Dépendance de version : Strictement réservé à PowerShell 7+ (absent par défaut de Windows PowerShell 5.1). 2. Risque de concurrence (<i>Thread Safety</i>) : Si plusieurs threads écrivent en même temps dans une variable partagée ou sur le même fichier texte sans verrouillage (<i>Mutex / Lock</i>), les données sont corrompues. 3. Débogage plus complexe : La capture et l'isolation des erreurs au sein de 20 threads concurrents nécessite une gestion rigoureuse des blocs try / catch.

4. Les Tâches d'Arrière-Plan Multi-Processus : Les Background Jobs (Start-Job)

Les **Background Jobs** représentent la technologie historique de traitement asynchrone de PowerShell, disponible de façon universelle sur toutes les versions du système (Windows PowerShell 2.0 à 5.1 et PowerShell 7+).

4.1 Fonctionnement interne : L'isolation par processus Windows

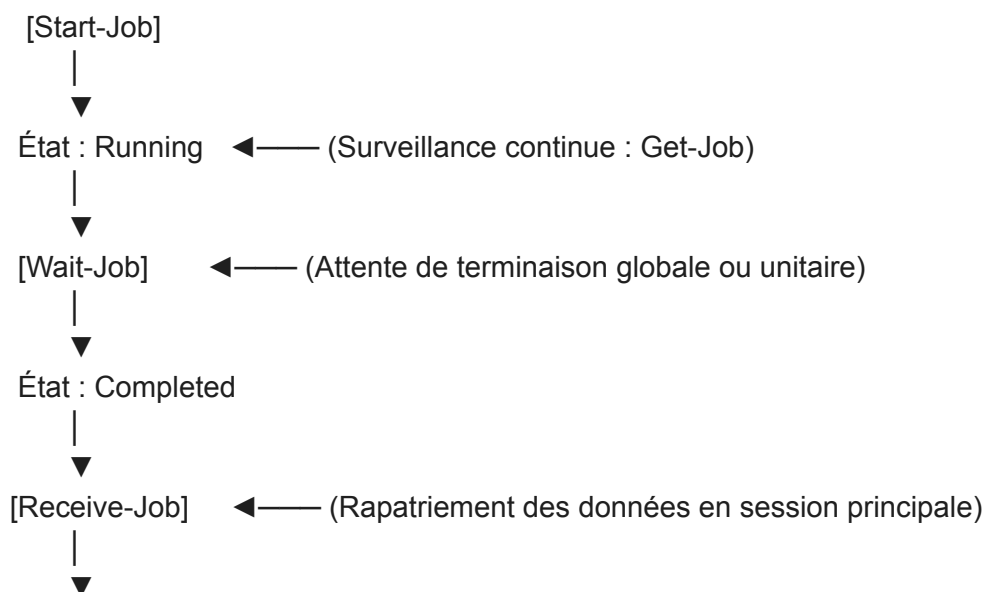
Contrairement au multithreading, un Job lancé via Start-Job ne crée pas un simple thread : **il instancie un processus système Windows complet et autonome (powershell.exe) en arrière-plan.**



Chaque sous-processus charge son propre moteur .NET, ses propres modules et sa propre mémoire.

4.2 Le cycle de vie rigoureux d'un Job

Le pilotage d'un Job repose sur ⁵ étapes séquentielles indispensables :



[Remove-Job] ←—— (Purge obligatoire de la mémoire vive)

Étape 1 : Démarrer la tâche (Start-Job)

```
$MonJob = Start-Job -Name "ScanStockage" -ScriptBlock {  
    param($Lecteur)  
    Start-Sleep -Seconds 3  
    Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DeviceID='$Lecteur:'"  
} -ArgumentList "C"
```

- **Le piège du passage d'arguments** : Le sous-processus ne partage aucune mémoire avec votre console. Le scope \$using: n'est pas supporté par Start-Job sous Windows PowerShell 5.1. Il faut obligatoirement transmettre les variables via **-ArgumentList** et les réceptionner avec **param()** au début du ScriptBlock.

Étape 2 : Surveiller l'état d'avancement (Get-Job)

Permet de relever l'Id, le Name et l'état (State) de la tâche :

- NotStarted : En file d'attente d'instanciation.
- Running : En cours d'exécution active sur le processeur.
- Completed : Terminé avec succès ; des objets attendent dans le tampon.
- Failed : Une erreur bloquante non interceptée a fait planter le code du Job.
- Stopped : Tâche interrompue prématurément par un administrateur (Stop-Job).

```
# Compter le nombre de jobs encore en activité  
(Get-Job -State Running).Count
```

Étape 3 : Synchroniser et bloquer (Wait-Job)

Si la suite du script a besoin des résultats pour consolider un rapport :

```
# Attendre la fin de tous les jobs de la collection  
$MesJobs | Wait-Job
```

```
# Attendre avec un délai maximum (Timeout de 15 secondes)  
Wait-Job -Name "ScanStockage" -Timeout 15
```

Étape 4 : Récolter les données et le piège du tampon (Receive-Job)

```
$Resultats = Receive-Job -Name "ScanStockage"
```

Attention au piège de la consommation du buffer : Par défaut, Receive-Job agit

comme une vidange de file d'attente (*Queue*). Dès que les données sont lues, le tampon mémoire du Job est **définitivement effacé**. Un second appel à Receive-Job renverra \$null.

Pour autoriser des lectures multiples sans vider la mémoire tampon, ajoutez impérativement le paramètre **-Keep** :

```
Receive-Job -Name "ScanStockage" -Keep
```

Étape 5 : Nettoyer la mémoire obligatoire (Remove-Job)

Contrairement à une variable classique nettoyée automatiquement par le ramasse-miettes (*Garbage Collector*), les jobs terminés **restent stockés indéfiniment en mémoire RAM** tant que la console PowerShell reste ouverte.

Dans un script d'audit régulier ou un traitement portant sur plusieurs dizaines de cibles, l'absence de purge génère une **fuite de mémoire majeure** (*Memory Leak*) :

Règle de nettoyage en fin de script :

```
Get-Job -State Completed, Failed, Stopped | Remove-Job
```

4.3 Avantages et Inconvénients de Start-Job

Caractéristique	Analyse technique
Avantages	1. Compatibilité universelle : Fonctionne nativement sur 100 % des machines Windows (PowerShell 2.0 à 7+). 2. Asynchronie totale immédiate : Rend instantanément la main à la console interactive. 3. Cloisonnement absolu : En cas d'effondrement mémoire ou de crash critique dans le job, la console de l'administrateur ne plante jamais. 4. Sécurité des variables : Aucun risque de corruption de mémoire entre les tâches (espaces d'adressage distincts).
Inconvénients	1. Empreinte mémoire massive :

	30 à 50 Mo de RAM par tâche (20 jobs $\approx$ à 1 Go de RAM monopolisée). 2. Latence d'initialisation élevée : Compter 1 à 2 secondes de délai rien que pour instancier le sous-processus. 3. Désérialisation des objets (CLIXML) : Les méthodes actives sur les objets rapatriés sont désactivées. 4. Lourdeur opérationnelle : Obligation d'écrire tout le cycle de récolte et de purge sous peine de fuite mémoire.
--	--

5. Comparatif Architectural Approfondi : Processus vs Threads

Pour concevoir des outils d'infrastructure professionnels, il est impératif de comprendre ce qui se passe physiquement dans le système d'exploitation :

[1. ARCHITECTURE START-JOB : MULTI-PROCESSUS]

Processus Principal (powershell.exe)

- |— Sous-processus 1 (powershell.exe) —► 35 Mo RAM / 1.5s démarrage
- |— Sous-processus 2 (powershell.exe) —► 35 Mo RAM / 1.5s démarrage
- |— Sous-processus 3 (powershell.exe) —► 35 Mo RAM / 1.5s démarrage

Total pour 20 jobs = ~700 Mo à 1 Go de RAM et pic CPU à l'initialisation.

[2. ARCHITECTURE FOREACH-OBJECT -PARALLEL : MULTI-THREAD]

Processus Unique (pwsh.exe)

- |— Thread 1 (Tâche A) —► ~1 Mo RAM / Démarrage quasi-instantané
- |— Thread 2 (Tâche B) —► ~1 Mo RAM / Démarrage quasi-instantané
- |— Thread 3 (Tâche C) —► ~1 Mo RAM / Démarrage quasi-instantané

Total pour 20 threads = ~20 Mo de RAM supplémentaires seulement.

Synthèse comparative

Critère d'évaluation	Jobs classiques (Start-Job)	Multithreading natif (ForEach-Object -Parallel)
Technologie système	Processus Windows indépendant (powershell.exe)	Threads managés .NET dans le processus hôte
Compatibilité système	Universelle (PowerShell 2.0 à 7+)	PowerShell 7+ uniquement (ou module ThreadJob en 5.1)
Consommation RAM	Lourde (~ par job)	Minime (~ par thread)
Latence au lancement	Lente (1 à 2 secondes par tâche)	Instantanée (<)
Passage de données	-ArgumentList et bloc param() obligatoire	Modificateur de scope \$using:NomVariable
Plafond d'extensibilité	Restreint (10 à 30 jobs simultanés max)	Très élevé (centaines de threads simultanés)
Nature des objets	Objets désérialisés (Deserialized.*)	Vrais objets .NET complets en mémoire

6. Régulation de la Charge : La Règle du ThrottleLimit

Lancer des opérations concurrentes sans restriction présente un risque critique : **l'effondrement des ressources de la machine hôte** (saturation CPU, épuisement de la mémoire ou engorgement de la carte réseau).

6.1 Le principe du pool glissant (*Worker Pool*)

Le paramètre **-ThrottleLimit** définit le nombre maximum de traitements simultanés autorisés à tourner en même temps :

```
1..50 | ForEach-Object -ThrottleLimit 5 -Parallel {
    Write-Host "Traitement du composant $_ sur le Thread
$([System.Threading.Thread]::CurrentThread.ManagedThreadId)"
    Start-Sleep -Seconds 1
```

}

- **Comportement du moteur** : PowerShell démarre les 5 premières tâches. Dès que l'une d'entre elles s'achève, la 6^e démarre immédiatement dans le thread libéré. Le nombre de tâches concurrentes sur le système ne dépasse jamais le seuil fixé.

6.2 Comment dimensionner son ThrottleLimit ?

Le réglage optimal dépend de la nature des opérations exécutées :

- **Traitements orientés réseau ou attentes (I/O-Bound)** :
(Exemples : ping de serveurs, requêtes API web REST, interrogation CIM distante, scan de ports).
Le processeur passe la majeure partie de son temps en attente des paquets réseau. Vous pouvez fixer un seuil de concurrence élevé : 20 à 50 .
- **Traitements orientés calcul processeur (CPU-Bound)** :
(Exemples : calculs de hachage SHA-256 de fichiers volumineux, compression d'archives zip, chiffrement).
Au-delà du nombre de cœurs physiques, les threads se disputent le temps de calcul et ralentissent le traitement global

7. Le Patron d'Architecture Industriel "Scatter-Gather" (Émettre - Récolter)

En production, l'architecture d'un script parallèle robuste repose sur le patron de conception

Scatter-Gather (Éparpiller pour exécuter, Rassembler pour synthétiser), découpé en 4 étapes fondamentales :

[Phase 1 : ÉMISSION (Scatter)]

Lancement des tâches non-bloquantes (Start-Job ou ForEach-Object -Parallel -AsJob)



[Phase 2 : SURVEILLANCE (Monitor)]

Boucle de contrôle active avec retour visuel (Write-Progress)



[Phase 3 : RÉCOLTE (Gather)]

Rapatriement consolidé de l'ensemble des flux d'objets (Receive-Job)



[Phase 4 : PURGE (Sanitize)]

Destruction systématique des résidus mémoire dans un bloc finally (Remove-Job)

```
finally {  
    # 4. Purge systématique (Sanitize)  
    Write-Progress -Activity "Audit Parallèle d'Infrastructure" -Completed  
    $Jobs | Remove-Job -Force  
}
```

8. Synthèse des Bonnes Pratiques de Production

1. **Toujours chronométrer pour vérifier le gain réel** : Lancer un job pour une action qui prend 5ms est contre-productif, car le coût d'initialisation du job 1 à 2 dépasse largement le temps de traitement de l'action.
2. **Ne jamais laisser de Jobs orphelins** : Tout script qui manipule des jobs doit systématiquement appeler Remove-Job au sein d'un bloc de clôture finally.
3. **Utiliser -Keep pendant le développement** : En phase de test dans la console, ajoutez toujours le paramètre -Keep à Receive-Job pour éviter de purger accidentellement vos données de test lors d'une première lecture.
4. **Adapter l'outil à la version de PowerShell en production** :
 - Privilégier systématiquement ForEach-Object -Parallel associé à -ThrottleLimit (ou avec -AsJob pour le non-bloquant).